



university of
 groningen

Potential-Based Reward Shaping for Planar Pushing

Single-Agent Control vs. Asymmetric Self-Play

Vlad Iftime s3426394

Faculty of Science and Engineering, Artificial Intelligence
University of Groningen

June 2026

1. Introduction & Research Questions

Outline



1. Introduction & Research Questions

2. Background & Related Work

reward shaping · curriculum & self-play · push mechanics & SE(2)

3. Problem Definition & MDP

4. Implementation

four models: PPO-PBRS, PPO-Curriculum, ASP-dPose, TASP-dPose

5. Evaluation

30 held-out scenes · 20 trials · Isaac vs gym

6. Results

7. Discussion

8. Conclusion & Takeaways

Research Questions



Three Research Questions

1. **RQ1:** Can potential-based reward shaping outperform hand-tuned dense rewards for goal reaching tasks in a robotic environment?
2. **RQ2:** Given a well-shaped PBRS reward function does adding a curriculum or asymmetric self-play improve final task success over a direct single-agent approach?
3. **RQ3:** Can a sufficiently informative reward (PBRS) *substitute* for an explicit curriculum in contact-rich pushing?

2. Background & Related Work



Related Work – Reward Shaping (PBRs)

Potential-based reward shaping (Ng, Harada & Russell 1999):

$$\mathcal{R}'(s, a, s') = \mathcal{R}(s, a, s') + \underbrace{\gamma\Phi(s') - \Phi(s)}_F$$

- **Policy invariance:** $Q'(s, a) = Q(s, a) - \Phi(s)$ – the optimal policy is unchanged for *any* state potential Φ .
- **Episodic validity** (Grześ 2017): $\gamma_{\text{shaping}}=1$ is safe – $\sum F = \Phi(s_T) - \Phi(s_0)$ is bounded.
- **Dynamic / arbitrary potentials** (Devlin & Kudenko 2012; Harutyunyan et al. 2015) – basis for our exponential Φ .

Why it matters here: lets us design a dense, well-behaved reward *without* changing the task's optimum – the foundation of every model in this thesis.

Related Work – Curriculum & Asymmetric Self-Play

Forced curricula

- **Surveys** (Narvekar et al. 2020; Portelas et al. 2020)
- **Reverse / goal** (Florensa et al. 2017): start near the goal, expand outward
- **Precision** (Luo et al. 2020): gradually tighten the success tolerance

Our use: a forced 3_{xx}phase pos→rot curriculum (PPO_{xx}Curriculum)

Automatic (self_{xx}play)

- **ASP** (Sukhbaatar et al. 2018; Plappert et al. 2021): Alice/Bob adversarial game + ABC
- **Time_{xx}based Alice** (Sukhbaatar 2018): $R_A \propto t_B - t_A$, self_{xx}regulating
- **Regret_{xx}based** (Dennis et al. 2020, PAIRED; Campero et al. 2021, AMIGo)

Our use: ASP_{xx}dPose (outcome) and TASP_{xx}dPose (time_{xx}based)

Related Work – Geometry & Mechanics of Planar Pushing

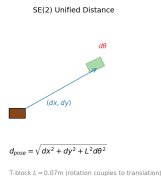
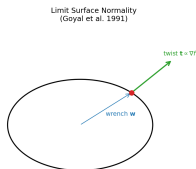


Push mechanics

- **Quasi-static pushing** (Mason 1986; Lynch & Mason 1996): motion set by the friction limit surface.
- **Limit-surface normality** (Goyal et al. 1991; Howe & Cutkosky 1996): object twist $\mathbf{t} \propto \nabla f(\mathbf{w})$ – every push couples translation and rotation.

SE(2) geometry

- **Lie-group pose distance** (Park 1995; Urain et al. 2023): a geodesic metric fusing translation and rotation with



3. Problem Definition & MDP

Planar-Pushing MDP – State, Action & Transition



State $s \in \mathbb{R}^{28}$: EE pose (6) | object pose (14) | goal pose (6) | errors (2)

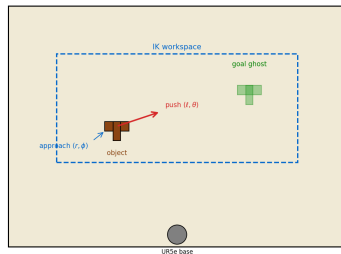
Action $a \in \mathbb{R}^4$ (object-relative): (r, ϕ, ℓ, θ) – 21 bins/dim, 194 481 discrete actions

Transition (Mason 1986): $t \propto \nabla f(\mathbf{w})$ – limit-surface normality couples **every** push with rotation

Success: pos < 5 cm **and** rot < 0.2 rad.

Validation: 30 scenes $\times$ 20 trials $\times$ up to 30 pushes

Planar Pushing Task — Object-Relative Action (r, ϕ, ℓ, θ)





Old Reward Formula – Why It Failed

Fractional Improvement Formula (Fixes P63-P64)

$$R = \underbrace{\alpha \cdot \frac{d_{\text{prev}} - d_{\text{now}}}{d_{\text{prev}}}}_{\text{position improvement}} + \underbrace{\alpha \cdot \frac{y_{\text{prev}} - y_{\text{now}}}{y_{\text{prev}}}}_{\text{rotation improvement}} - \underbrace{\beta_{\text{pos}} \cdot d_{\text{now}}}_{\text{position penalty}} - \underbrace{\beta_{\text{rot}} \cdot y_{\text{now}}}_{\text{rotation penalty}}$$

$$\alpha = 3.0, \beta_{\text{pos}} = 0.5, \beta_{\text{rot}} = 0.25$$

$d_{\text{prev}}, d_{\text{now}}$: position error before/after push; $y_{\text{prev}}, y_{\text{now}}$: rotation error before/after push

Problem 1 – Non-Markov: Depends on d_{prev} – same state transition gets different reward from different starting positions. Value function ill-defined.

Result: The fractional formula suffers from two structural flaws – non-Markov dependence and

Problem 2 – Gradient Starvation: 1cm improvement from 30cm:

$\alpha \cdot 0.01 / 0.30 - \beta_{\text{pos}} \cdot 0.29 = -0.045$. Progress gets **negative** reward. $\mathbb{E}[R] \approx -0.15$, near-zero variance.

4. Implementation



The Four Models at a Glance

Model	Idea	Reward / curriculum
PPO-PBRS	single agent, no curriculum	two-potential PBRS (pos, rot)
PPO-Curriculum	single agent + staged curriculum	PBRS + forced pos→rot
ASP-dPose	self-play (Alice/Bob)	SE(2) PBRS, outcome Alice (+5/ - 1)
TASP-dPose	self-play (Alice/Bob)	SE(2) PBRS, time-based Alice

All four share the same PPO+LSTM backbone, object-relative push primitive, 528 envs, and ~3000 iterations. They differ only in **reward shape** and **training-goal generation**.

[Schulman et al. 2017] [Ng et al. 1999] [Sukhbaatar et al. 2018]



PPO-PBRS – Single-Agent (Our Best Model)

Architecture: Single PPO agent, LSTM(28→256) + MLP[512, 256] trunk, MultiCategorical head (4 dims $\times$ 21 bins). No curriculum, no Alice/Bob, no GoalEncoder.

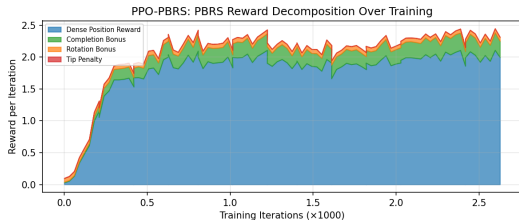
Training: 528 envs, 15 pushes/rollout, ~ 3000 iterations ~ 1.6 days wall-clock (~ 0.022 it/s).

Why it works:

1. **Dense, bounded gradient** – PBRS fires on every push
2. **Fixed goal distribution** – stationary random goals; no adversarial shift
3. **Single network** – one agent, one optimizer; no multi-agent instability

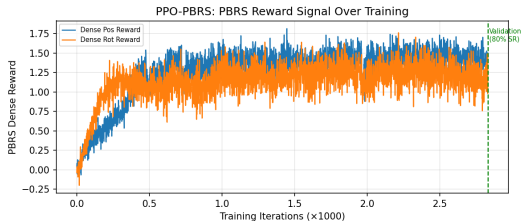


PBRS – Reward Signal Analysis



Reward diet: $\sim 85\%$ dense PBRS shaping,
+5/+2 sparse bonuses, penalties.

Dense signal on every push, bounded rewards, stable value – PBRS avoids the gradient starvation and critic instability of the old fractional formula.



Signal vs performance: bounded exponential
potentials $\rightarrow$ stable, correlated gradient.

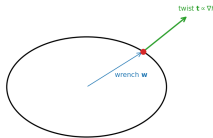
SE(2) Unified Distance – Self-Play Variants

SE(2) metric: $d_{\text{pose}} = \sqrt{dx^2 + dy^2 + L^2 d\theta^2}$

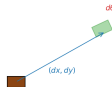
- Single scalar combining (dx, dy) and $d\theta$; L = char. length (T-block 0.07 m)
- **Single-potential PBRs:** $\Phi(s) = w e^{-k d_{\text{pose}}^2}$, $k = 30$, $w = 10$
- Eliminates competing pos/rot gradients
 - aligns reward with the coupled physics

Result: SE(2) does *not* rescue self-play – ASP-dPose reaches only 6.7% validation SR vs single-agent 80%. The self-play curriculum is the root cause, not the metric.

Limit Surface Normality
(Goyal et al. 1991)



SE(2) Unified Distance



$$d_{\text{pose}} = \sqrt{dx^2 + dy^2 + L^2 d\theta^2}$$

T-block $L = 0.07\text{m}$ (rotation couples to translation)

PPO-Curriculum – Position→Rotation Curriculum



Design: Same PPO agent as PPO-PBRS + 3-phase curriculum:

1. $w_{\text{rot}} = 0$, position-only training
2. Triggered when episodic position-SR ≥ 0.5 for 20 iters: w_{rot} ramps $0 \rightarrow 10$ over 200 iters
3. Full PBRS (multi-objective)

Original trigger was mis-specified: the first version used a per-push mean position error threshold of 0.08m – below even the *best* model's validation PosErr (0.202m). The curriculum never activated because the threshold was unreachable by construction (P82 fix).

Corrected trigger: gates on episodic position-only SR ≥ 0.5 over a 20-iter lookback. Curriculum now activates and reaches **76.7% SR** – only 3.3pp below PPO-PBRS.

Self-Play – ASP / TASP Architecture (Alice/Bob)



Alice (Goal Proposer):

- PPO agent, free-roaming push phase – leaves T-block in a new SE(2) pose
- That pose becomes Bob's target goal
- Reward: +5 if Bob fails to reproduce it, -1 if Bob succeeds

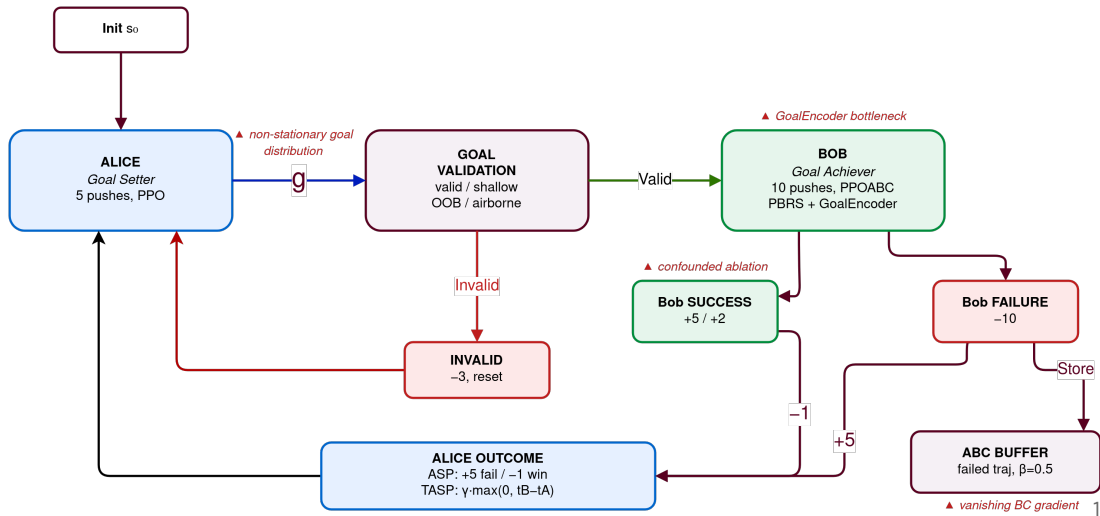
Bob (Goal Achiever):

- PPOABC + GoalEncoder – must reproduce Alice's goal pose from scratch
- Reward: PBRS dense shaping + sparse completion bonuses (+5/+2)
- Failed trajectories stored in ABC buffer for behavioral cloning ($\beta_{abc}=0.5$)

Two variants: both use the SE(2) unified d_{pose} metric (T-block); they differ only in Alice's reward – **ASP-dPose** (outcome-based) vs **TASP-dPose** (time-based). **Excluded:** base ASP (separate pos/rot potentials) and the GoalEncoder-ablation – no matched-protocol Isaac validation.



ASP Loop – Alice $\leftrightarrow$ Bob Cycle





ASP-dPose vs TASP-dPose – Alice Reward

Both share the SE(2) d_{pose} Bob reward and T-block object; only Alice's reward differs.

Self-play variant	Alice reward	Valid SR
ASP-dPose (outcome-based)	$R_A = +5 \text{ fail} / -1 \text{ win}, \beta_{\text{abc}}=0.5$	6.7%
TASP-dPose (time-based)	$R_A = \gamma_{\text{sp}} \cdot \max(0, t_B - t_A), \text{no ABC}$	16.7%

Findings:

- Time-based Alice (TASP-dPose) outperforms outcome-based (ASP-dPose) by +10pp
- TASP-dPose disables ABC – the gain comes from the time-based reward alone
- Best self-play variant (TASP-dPose, 16.7%) still **4.8** $\times$ below single-agent (PPO-PBRS, 80.0%)

5. Evaluation

Validation Test Suite – 30 Held-Out Scenes



Validation Test Suite – 30 Held-Out T-block Scenes (start → goal)



Each cell is one held-out scene: T-block **start pose** → **goal pose**. Rows: rotation (T1-T10), position-only (T11-T20), position+rotation (T21-T30). Every model is evaluated on this identical suite (20 trials × up to 30 pushes per scene).

[Mason 1986] [Lynch & Mason 1996]



Isaac Lab – the Right Venue for Self-Play

Why GPU-batched simulation is necessary for this comparison:

Throughput (push-macros/s, measured):

Three pillars:

Setting	push/s	1M pushes
Isaac (1 GPU, 528 env)	172	~1.6 h
gym single-agent (6 CPU)	91	~3.0 h
gym self-play (1 CPU)	22.4	~12.4 h

Self-play on GPU vs CPU:

- GPU: self-play & single-agent identical throughput (~ 0.022 it/s)
- CPU: self-play forced single-process $\rightarrow \sim 7.7\times$ slower than single-agent

1. **GPU-batched parallelism**

528 envs/step $\rightarrow$ 7,920-sample PPO batch on one GPU. The non-stationary self-play objective needs a large batch for stable gradients.

2. **Robotic fidelity**

UR5e + 3D PhysX contact + cuRobo IK = the actual SE(2) task. gym-pusht is a 2D abstraction (diagnostic only).

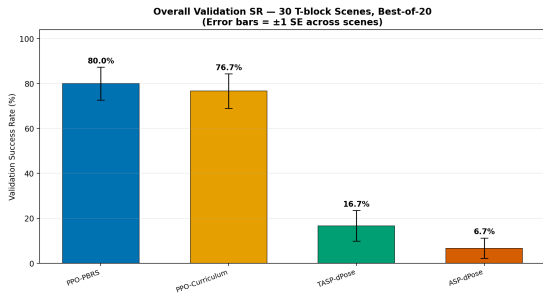
3. **Time-to-scale (self-play)**

Self-play reaches ~ 10 M training pushes in ~ 16 h on GPU vs ~ 5 days on CPU. The ²³

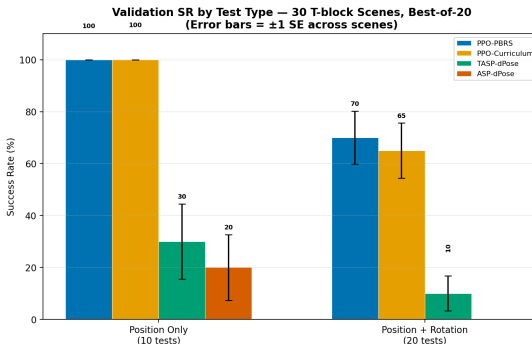
6. Results



Multi-Model Validation



Overall SR & SE — 30 T-block scenes, best-of-20

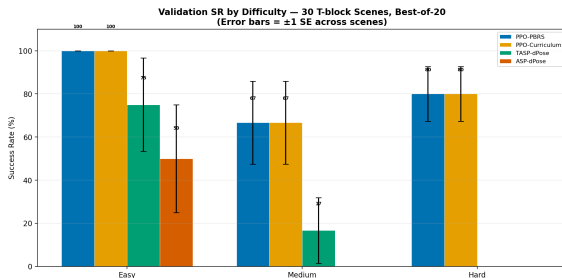


SR by test type: pos-only (10 tests) vs pos+rot (20 tests)

PPO-PBRS vs PPO-Curriculum: 80.0% vs 76.7% – 3.3pp gap **not statistically significant**

($p > 0.05$ on 30 independent scenes). **Self-play (ASP-dPose, TASP-dPose):** 4.8–11.9 $\times$ below single-agent – gap is significant. *Error bars = ± 1 standard error across scenes. Base ASP and the*

Comparison Summary – All Models with Error Bars



Key numbers (30 T-block scenes):

Model	SR	SE	95% CI
PPO-PBRS	80.0%	7.3%	65–95%
PPO-Curriculum	76.7%	7.7%	61–92%
TASP-dPose	16.7%	6.8%	3–30%
ASP-dPose	6.7%	4.6%	0–16%

PPO-PBRS vs PPO-Curriculum gap: 3.3pp – not

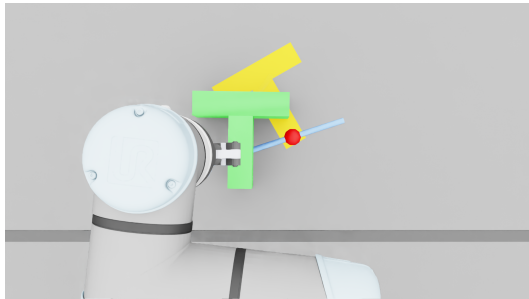
statistically significant ($p > 0.05$). Both solve the task similarly. Self-play variants are $4.8\text{--}11.9\times$ below – this gap **is** significant.

Error bars = ± 1 standard error across scenes. Base ASP and the GoalEncoder-ablation are excluded – neither has a matched-protocol Isaac validation. See thesis appendix.

PPO-PBRS – Validation Results

30 held-out T-block scenes (20 trials each):

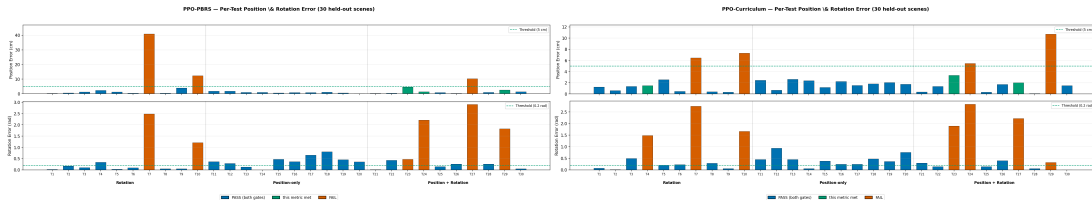
- **80.0% SR** (24/30) – pos-only **100%**, pos+rot **70%**
- PosErr **0.032 m**, RotErr **0.568 rad** (mean over all 30 scenes, incl. failures)
- Per-attempt (trial-level) SR **66%**; scene SR counts a scene solved if any of its 20 trials passes



Overhead view – PPO-PBRS solving a pos+rot scene ▶

[play clip](#)

Results – Per-Test Error (Single-Agent Models)

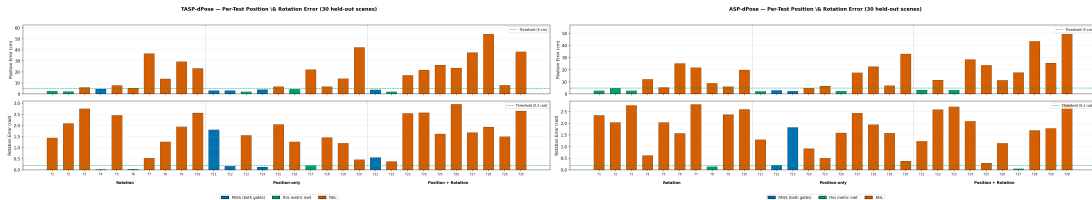


PPO-PBRS – 80.0% (24/30)

PPO-Curriculum – 76.7% (23/30)

Both clear position on nearly every scene; the failures are **rotation**-limited (red bars, bottom) on the hardest pos+rot scenes.

Results – Per-Test Error (Self-Play Models)

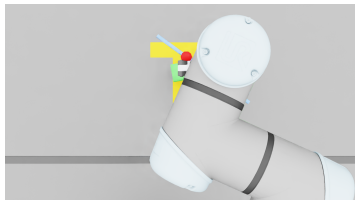


TASP-dPose – 16.7% (5/30)

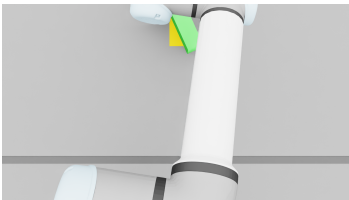
ASP-dPose – 6.7% (2/30)

Self-play fails on *both* axes across almost all scenes – rotation error stays > 1.4 rad. The few successes are easy position-only scenes.

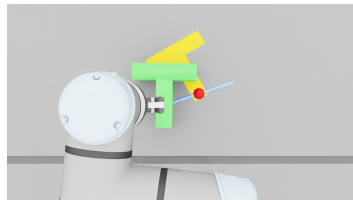
PPO-PBRS – Validation Runs (overhead)



Forward (pos-only) ▶ [play clip](#)



Lateral push (pos-only) ▶ [play clip](#)

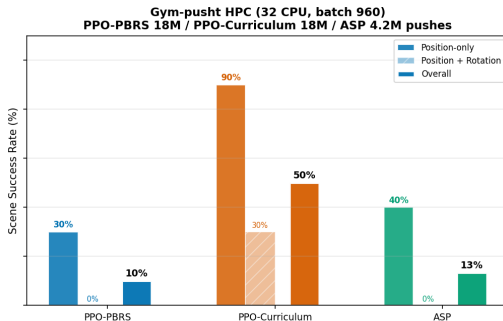
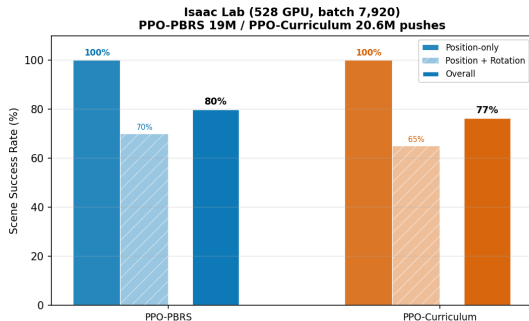


Translate + rotate ▶ [play clip](#)



Cross-Environment Validation – Isaac vs Gym

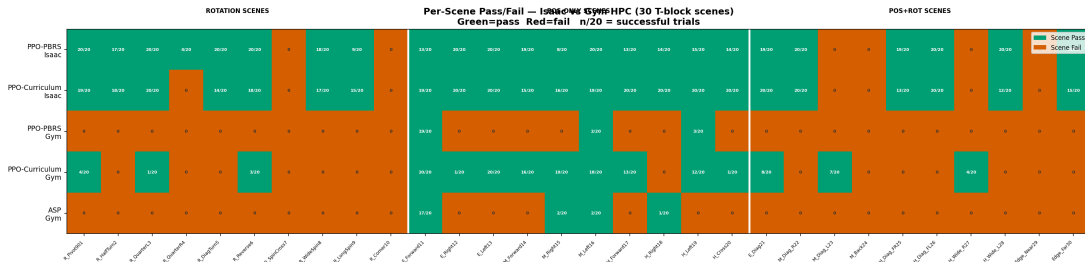
Cross-Environment Validation SR — Isaac Lab vs Gym-pusht HPC
Same thesis gate, same 30 T-block scenes, same total push budget (~18-20M for PPO-PBRS / PPO-Curriculum)



The ordering single-agent $\approx$ curriculum $\gg$ self-play holds in both simulators. Gaps compress in the simpler gym env: PPO-Curriculum benefits most from the small-batch regime (50% gym vs 76.7% Isaac), consistent with the RQ3 batch-size caveat.



Per-Scene Outcomes – Isaac vs Gym



Green = scene solved (any of 20 trials), red = failed; cell text = successful trials / 20. The hardest pos+rot scenes (right block) defeat every configuration except the two single-agent models on Isaac. Self-play (gym) solves only a few easy position-only scenes.

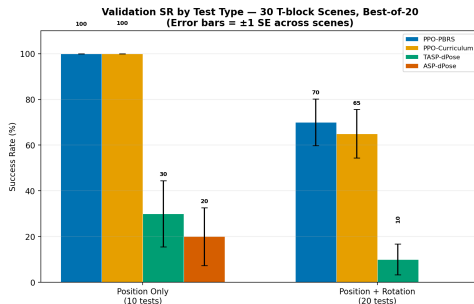
7. Discussion

Discussion – What the Validation Suite Reveals



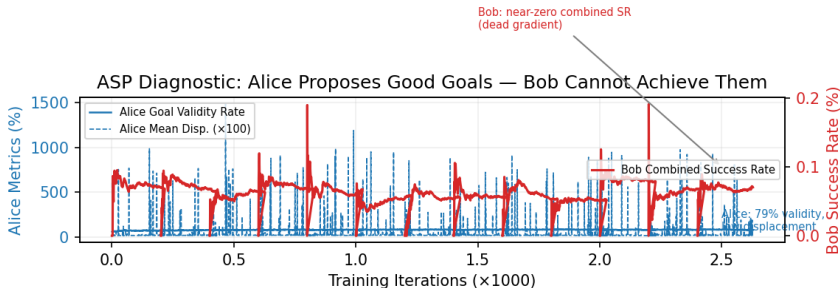
The combined pos+rot gate is the wall:

- Position-only scenes (T11–T20): solved by both single-agent models (100%).
- Pos+rot scenes (T1–T10, T21–T30): the bottleneck – PPO-PBRS 70%, self-play $\leq 10\%$.
- Failures are almost always **rotation**-limited, not position – consistent with the coupled limit-surface physics.
- Hardest scenes are large-yaw / wide-translation (e.g. R_SpinCross, H_Wide): they need many coordinated





ASP Diagnostic – Alice vs. Bob



Alice proposes valid, low-displacement goals, yet Bob cannot achieve the combined pos+rot objective under the non-stationary adversarial goal distribution.



Why ASP Fails – Failure Modes

1. **Non-stationary goal distribution:**

- Alice learns continuously → Bob's training distribution shifts every iteration
- PPO's importance ratio computed against stale rollouts from a previous Alice
- Distribution shift dominates the ratio – PPO clip fires on most transitions

2. **Confounded ablation (C6):**

- ASP changes 3 axes at once: goal distribution, agent count, episode budget
- The collapse could be due to harder/non-stationary goals, not the curriculum mechanism itself
- Time-based self-play (TASP-dPose, 16.7%) shows partial rescue – self-regulating curriculum helps



Why ASP Fails – The Evidence

What we observe:

- Outcome-based self-play (ASP-dPose): 6.7% – essentially random
- Time-based self-play (TASP-dPose): 16.7% – partial rescue, +10pp
- Both self-play variants have $\text{RotErr} > 1.4$ rad – rotation **never learned**
- Even the best variant (TASP-dPose) is $4.8\times$ below single-agent

Why this is not just a budget issue:

- All models trained for 3000 iters at 528 envs on identical hardware
- ASP and single-agent have identical per-iteration wall-clock (~ 0.022 it/s)
- The gap is **structural**, not due to compute starvation

Combined effect: The non-stationary goal distribution + multi-axis design confounds prevent Bob from learning the combined pos+rot objective. Time-based Alice partially mitigates the distribution shift but cannot close the gap to single-agent performance.



Caveat – Confounded Ablation

Problem: the self-play variants differ from single-agent PPO-PBRS on *at least 4 axes simultaneously*:

Variable	PPO-PBRS (80.0%)	Self-play (6.7–16.7%)
Agent count	1	2 (Alice + Bob)
Goal distribution	Fixed random	Adversarial, shifting
GoalEncoder	None	ϕ -MLP 8D latent
ABC buffer	None	Yes (ASP-dPose) / No (TASP-dPose)
Historical pool	None	Yes
Episode budget	15 pushes/agent	5 (Alice) + 10 (Bob)
PPO updates/iter	1	2 (Alice, then Bob)

The $4.8\times$ gap cannot be attributed to any single lever.

- TASP-dPose (time-based Alice, no ABC) vs ASP-dPose (outcome-based, ABC enabled) isolates the Alice reward axis – TASP-dPose gains +10pp
- But this is a *single* 1-axis experiment within a 7-axis package change



Key Result – PBRs Substitutes for Curriculum

Model	Valid SR	vs Single-Agent
PPO-PBRs (single-agent)	80.0%	– (baseline)
PPO-Curriculum	76.7%	–3.3pp (n.s.)
TASP-dPose (time-based self-play)	16.7%	4.8× gap
ASP-dPose (outcome self-play)	6.7%	11.9× gap

The Substitution Finding (RQ3)

- **Curriculum adds no value at this scale:** PPO-Curriculum (76.7%) trails PPO-PBRs (80.0%) by 3.3pp, but this gap is **not statistically significant** ($p > 0.05$). A sufficiently informative reward (PBRs) *substitutes* for the staged curriculum.
- **Scope / caveat:** substitution holds when the PPO batch is large enough for stable gradients (Isaac, batch 7,920). At small batch (gym, batch 960) the curriculum *does* help, – PPO-Curriculum 50% vs PPO-PBRs 10%. So the substitution is regime-dependent



What We Found – Theory vs Practice

Self-play did *not* yield a useful curriculum in this setting

In this contact-rich, IK-gated, multi-objective SE(2) pushing task under a single-GPU budget, our self-play implementation produced 6.7–16.7% validation SR vs 80.0% for single-agent. The adversarial mechanism did not generate a learnable progression of goals. This does *not* refute self-play in general – it demonstrates a failure mode under realistic robotics constraints.

The bottleneck is the goal *distribution*, not the goal *representation*

Bob's individual position and rotation skills are partly learned, but the combined gate rarely fires under the non-stationary adversarial goal distribution. The failure sits at the exploration / training-distribution level, not the observation encoding.



What We Found – Theory vs Practice

Forced curriculum: functional but not beneficial

PPO-Curriculum's original trigger (per-push mean PosErr $< 0.08m$) was mis-specified – the threshold was unreachable by construction. A corrected trigger (episodic position-SR ≥ 0.5) produces a competent model (76.7%), but the staged pos→rot curriculum does *not* beat the simpler no-curriculum baseline (80.0%). Curriculum works – it just doesn't add value at this batch size.

PBRs theory strongly supported (Ng et al. 1999; Grześ 2017)

The policy invariance theorem holds in practice, even in contact-rich manipulation with stochastic PhysX physics. PBRs provides clean, bounded, Markovian rewards. Parameters $k_p=30$, $k_r=5$, $w=10.0$ are robust across all single-agent runs.

8. Conclusion & Takeaways

Conclusion



1. PBRs solves planar pushing (RQ1)

PPO-PBRs achieves **80.0% validation SR** on 30 held-out T-block scenes – 100% pos-only, 70% pos+rot – with a simple single-agent PPO architecture and no curriculum. PBRs preserves the optimal policy while providing dense, bounded, Markovian gradients. This is the **recommended baseline**.

2. Self-play does not improve performance in this domain (RQ2)

Both self-play variants score 6.7–16.7%, a $4.8\text{--}11.9\times$ gap from single-agent. Time-based Alice (TASP-dPose, 16.7%) partially rescues the adversarial dynamics but cannot close the gap. Complexity adds no benefit.

Conclusion



3. PBRs can substitute for a curriculum (RQ3)

A well-shaped reward makes the staged curriculum redundant at this scale: PPO-Curriculum (76.7%) is statistically tied with PPO-PBRs (80.0%). The substitution is regime-dependent – at small PPO batch the curriculum does help (gym: 50% vs 10%). Self-play (6.7–16.7%) helps in neither regime.

Start simple. PPO-PBRs single-agent at 528 envs trains in ~ 1.6 days on an RTX Pro 6000 and achieves 80% SR. Add complexity only when proven to help – and expect it usually won't.



Limitations & Research Gaps

What we couldn't test (training budget limited to 3000 iters $\times$ 528 envs):

- **TASP-dPose-BobPenalty:** Bob time penalty $R_B \leftarrow -\gamma_{sp} \cdot t_B$ (full Sukhbaatar symmetric reward). Could create zero-sum dynamics that stabilize self-play training
- **Time-based self-play + ABC:** time-based Alice with ABC enabled ($\beta_{abc} = 0.5$). ABC might boot after Alice converges
- Larger models (Transformer, diffusion policy), sim-to-real transfer (physical UR5e)

Remaining literature gaps (from systematic survey of 81 papers):

- No paper on SE(2) metric design specifically for RL reward shaping
- No paper on exploiting continuous rotational symmetry (SO(2)) in object manipulation RL
- No formal convergence analysis of effort-based self-play rewards



Future Work – Promising Directions

1. **Zero-sum time-based self-play (TASP-dPose-BobPenalty):** Bob penalty creates near-zero-sum game. From differentiable game theory (Letcher et al. 2019), zero-sum games have convergent (Hamiltonian) dynamics – this could stabilize self-play where general-sum diverges
2. **ABC after Alice convergence:** Currently ABC fails because Alice acts randomly ($bc_ratio \approx 1.0$). If Alice converges first, Bob starts with a meaningful imitation target
3. **Relaxed combined success gate:** The combined pos+rot gate is the bottleneck (70% single-agent vs 0–10% for self-play). Partial credit for one objective could provide gradient to learn both
4. **PAIRED (Dennis et al. 2020):** Antagonist minimizes protagonist's regret rather than maximizing difficulty – solvable but challenging environments, addressing the exact failure mode

Computational Overhead – Self-Play Is Not Slower

Controlled measurement (same machine, same 528 envs, same cuRobo IK + physics):

PPO-PBRS (single-agent) vs the self-play variants run at $\sim 1.0\times$ per-iteration wall-clock (~ 0.022 it/s for both). The 2-agent machinery (Alice + Bob, GoalEncoder, ABC, historical pool) adds **no** measurable per-iteration overhead – the shared cuRobo IK + PhysX contact step dominates the cost.

Key result:

- Self-play is **not** slower per iteration – it simply **does not learn**
- The real cost of self-play is model/code complexity and added failure modes (two networks, non-stationary objective, ABC), *not* throughput
- So the $4.8\times$ performance gap is **not** a compute-budget artifact – both used the same compute

Extra Takeaway – ManagerBasedRL vs DirectRL



An engineering observation, separate from the scientific claims.

- Isaac Lab's **ManagerBasedRLEnv** (Action/Observation/Event/Termination managers) adds per-step Python dispatch; **DirectRLEnv** bypasses it with direct tensor ops.
- This is an *architecture* cost, independent of ASP – self-play and single-agent run at the same per-iteration time ($\sim 1.0\times$).

API	env-steps/s	relative
DirectRLEnv	[to measure]	[run pending]
ManagerBasedRLEnv	[to measure]	[run pending]

Controlled same-task measurement to be run; numbers will replace the placeholders. No multiplier is claimed until then.



university of
 groningen

Thank You – Questions?

Vlad Iftime s3426394

Faculty of Science and Engineering, Artificial Intelligence
University of Groningen

June 2026



Appendix Outline

- A. Push Mechanics – Limit Surface & Characteristic Length L
- B. Training curves for all PBRs models (Success Rate, Reward, Loss)
- C. PBRs parameter sensitivity (k_p , k_r , w_{pos} , w_{rot})
- D. Validation scene descriptions (30 test cases)
- E. cuRobo IK pipeline and workspace configuration
- F. Observation space specifications (all model variants)
- G. Per-test error bars – PPO-PBRs vs PPO-Curriculum head-to-head and all-model comparison (95% binomial CIs)



PBRS – The Shaping Theorem

Ng et al. (1999):

$$\mathcal{R}'(s, a, s') = \mathcal{R}(s, a, s') + \underbrace{\gamma\Phi(s') - \Phi(s)}_{\text{shaping reward } F}$$

Policy invariance: $Q'(s, a) = Q(s, a) - \Phi(s)$ – optimal actions unchanged.

Our potential functions (PPO-PBRS / PPO-Curriculum):

$$\Phi(s) = w_{\text{pos}} e^{-k_p d_{\text{pos}}^2} + w_{\text{rot}} e^{-k_r c(s)}, \quad c(s) = \frac{1 - \cos(\theta^* - \theta)}{2}$$

$k_p=30, k_r=5, w_{\text{pos}}=w_{\text{rot}}=10.0$

Dense reward: $F(s, s') = \Phi(s') - \Phi(s)$ – difference only, no constant drain.

Episodic PBRS (Grześ 2017): $\gamma_{\text{shaping}}=1.0$ valid – $\sum F = \Phi(s_T) - \Phi(s_0)$ bounded.



PBRs – Why It's the Right Choice

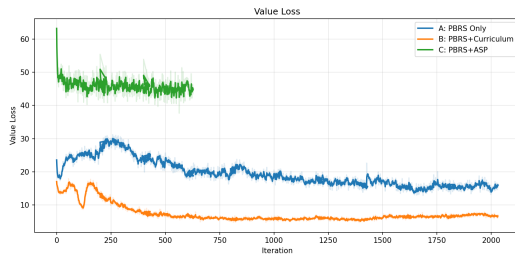
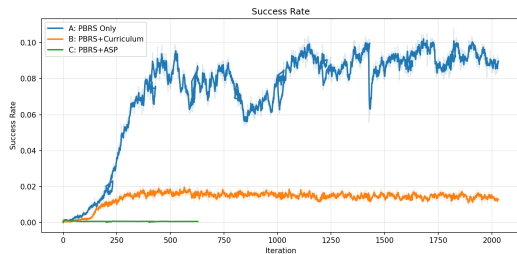
Properties of $F(s, s') = \Phi(s') - \Phi(s)$:

1. **Markovian** – depends only on s, s' , not history.
2. **Bounded** – exponential potentials prevent reward explosions from PhysX contact spikes.
3. **Zero-sum cycles** – detour pushes net to zero; agent free to explore without reward hacking.
4. **No constant drain** – $F=0$ when stationary; no penalty for hesitation or deliberation.

Key consequence: you can replace any ad-hoc reward with a potential-based one *without changing what the agent learns* – only *how fast* it learns. Reward design and policy optimisation are decoupled.



PPO-PBRS – Training Dynamics



Left: PPO-PBRS (blue) steadily improves training success rate over 3000 iterations. PPO-Curriculum (orange) follows a similar trajectory but converges slightly lower. Self-play variants remain near baseline throughout. **Right:** Stable value function – no explosion. Initial $V \approx 0.057$ with gain= 0.01.



PPO-Curriculum vs PPO-PBRS

Metric	PPO-PBRS	PPO-Curriculum	Gap
Validation SR	80.0%	76.7%	1.04×
PosErr (m)	0.032	0.023	B 28% lower
RotErr (rad)	0.568	0.663	A 14% lower
Pos-only SR	100%	100%	equal
Pos+rot SR	70%	65%	5pp

Result: Curriculum improves position precision (0.023 vs 0.032 m) from the pos-only phase, but slightly degrades rotation (0.663 vs 0.568 rad). Net effect: B is 3.3pp behind A. **Statistical note:** The 3.3pp A vs B gap is **not** significant ($p > 0.05$, two-tailed z-test; SE of difference = ± 10.6 pp across 30 independent scenes). Both A and B solve the task similarly well. **Takeaway:** A staged curriculum *can* work alongside PBRS, but does not beat the simpler no-curriculum baseline.



Self-Play – Validation Comparison

Model	Valid SR	PosErr	RotErr
PPO-PBRS (single-agent, ref.)	80.0%	0.032 m	0.568 rad
TASP-dPose (time-based)	16.7%	0.158 m	1.457 rad
ASP-dPose (outcome-based)	6.7%	0.143 m	1.612 rad

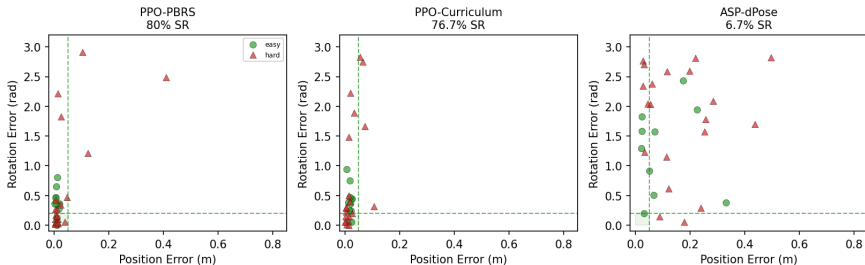
Key findings:

- Time-based self-play (TASP-dPose) is $2.5\times$ better than outcome-based (ASP-dPose), but still **$4.8\times$ below** single-agent PPO-PBRS
- No self-play variant achieves pos+rot SR above 10% – the combined gate remains the bottleneck
- Self-play complexity provides no benefit



Comparison Summary – Per_{xxx}Scene Position × Rotation Error

Validation Failure Taxonomy — Success Rectangle (pos<5cm, rot<0.2rad)



Three observations from the success rectangle (pos<5 cm, rot<0.2 rad):

- **PPO-PBR5** (left): 24/30 scenes land inside the green box – solves position and rotation on nearly all scenes.

Appendix A – Push Mechanics: Limit Surface & L

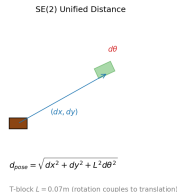
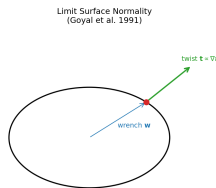


Limit surface normality (Goyal 1991):

$t \propto \nabla f(\mathbf{w})$ – twist is normal to the surface, so every push couples translation *and* rotation.

Characteristic length L (radius of gyration):

- **T-block** $L = 0.07$ m – an off-centre push couples translation *and* rotation
- L is a physical constant, *not* a hyperparameter

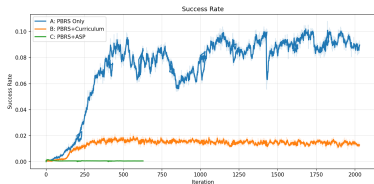


The T-block rotates under an off-centre push (coupled SE(2) motion)

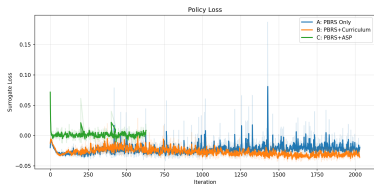
Appendix B – Training Curves (All PBRs Models)



Success Rate



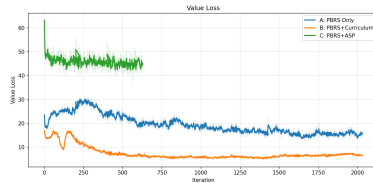
Policy Loss



Mean Reward



Value Loss

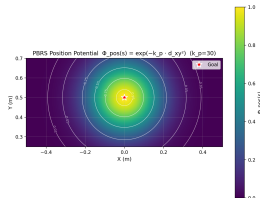


Appendix C – PBRs Theory Analysis



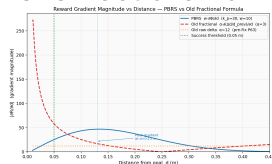
Potential Landscape

$$\Phi(s) = w \cdot \exp(-k \cdot d^2)$$

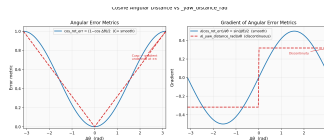


Gradient Comparison

PBRs vs Fractional Formula



Cosine Angular Distance $c(s) = (1 - \cos(\theta^* - \theta))/2$



Appendix G – Per-Test Error Bars (95% Binomial CIs)

